



assignment 4 , matlab

Linear Control Systems

Department of Electrical Engineering

Student: Hamed Batani 402101339

Instructor: Prof. Ahi

1 a : Breakaway/Break-in Candidate Points (Root Locus)

1.1 Problem Statement

Write a MATLAB function that takes a transfer function in the form

$$G(s) = \frac{a(s)}{b(s)}$$

and returns the candidate breakaway/break-in points of the root locus. Verify the results by plotting the root locus using the `rlocus` command for the example transfer functions.

1.2 Method / Formula Used

For a unity-feedback root locus, the gain can be written as

$$K(s) = -\frac{b(s)}{a(s)}.$$

Breakaway/break-in candidate points occur at extrema of $K(s)$ along the real axis, hence they satisfy

$$\frac{dK}{ds} = 0.$$

Using $K(s) = -\frac{b(s)}{a(s)}$ and the quotient rule:

$$\frac{d}{ds} \left(-\frac{b(s)}{a(s)} \right) = 0 \iff a(s) \frac{db(s)}{ds} - b(s) \frac{da(s)}{ds} = 0.$$

Therefore, the candidate points are the solutions of

$$a(s)b'(s) - b(s)a'(s) = 0.$$

1.3 MATLAB Implementation

```
clear; clc; close all;

s = tf('s');

% Define G(s)
G = (s^2) / ((s^2 - 1)^2);

candidates = breakaway_candidates(G);
disp(candidates)

% root locus for verification
figure; rlocus(G); grid on;

function candidates = breakaway_candidates(G)
    G = tf(G);
    [num, den] = tfdata(G, 'v');

    syms s
    a = poly2sym(num,s);
    b = poly2sym(den,s);

    eq = a*diff(b,s) - b*diff(a,s);

    candidates = roots(sym2poly(eq));
end
```

1.4 Results and Verification Figures

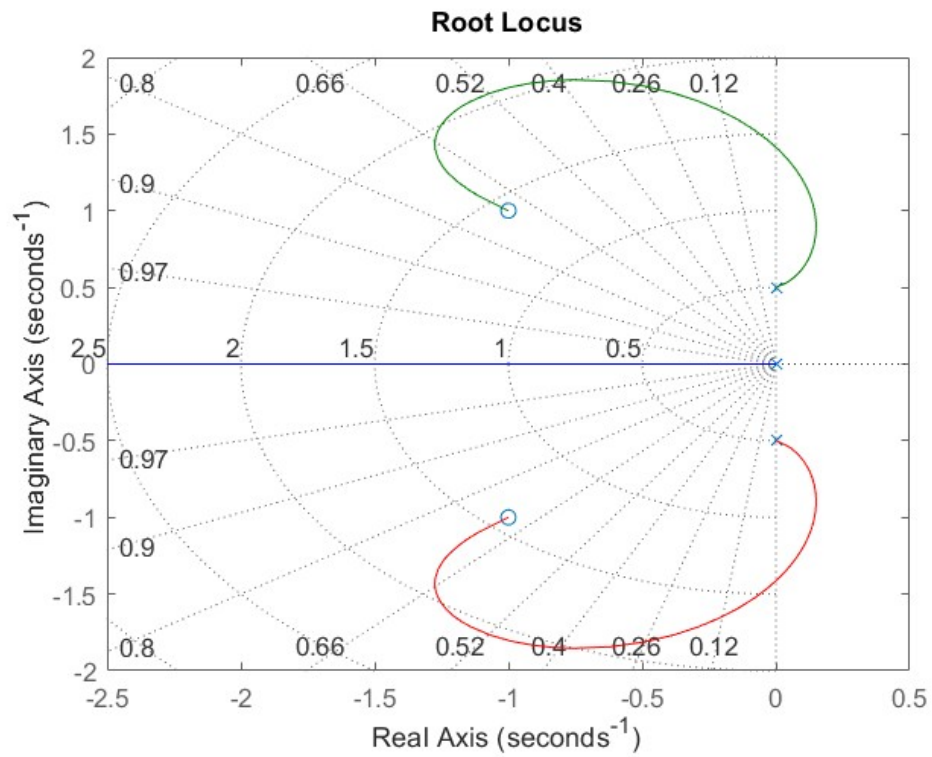


Figure 1: Root locus plot for $G_1(s)$.

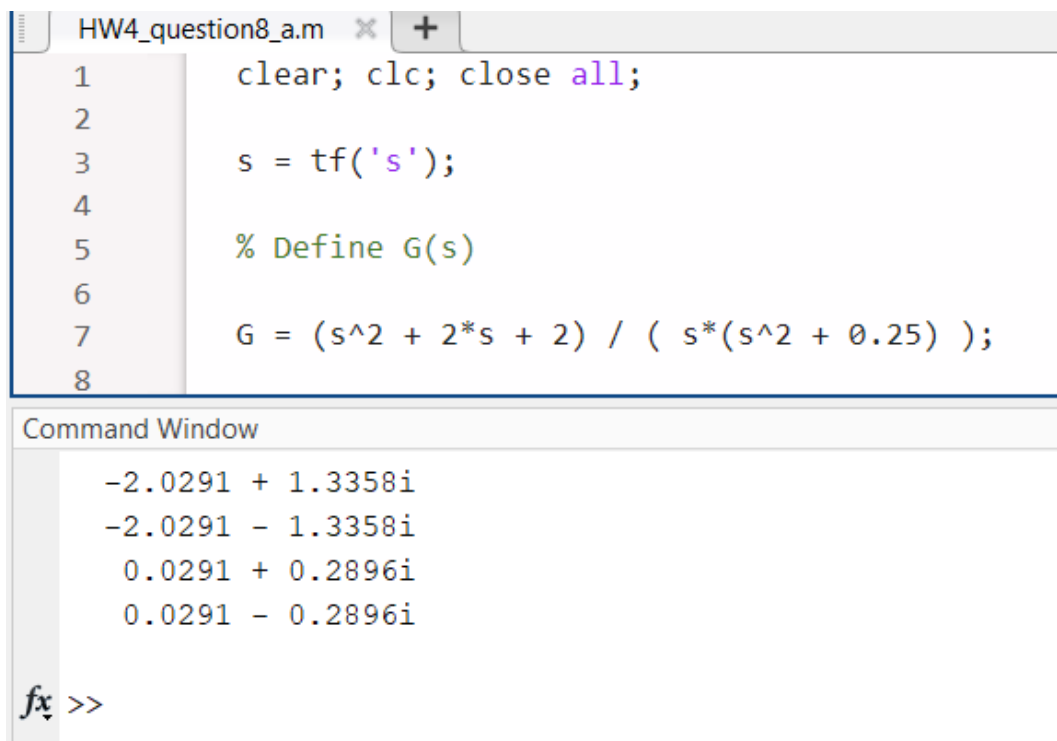


Figure 2: Candidate breakaway/break-in points for $G_1(s)$.

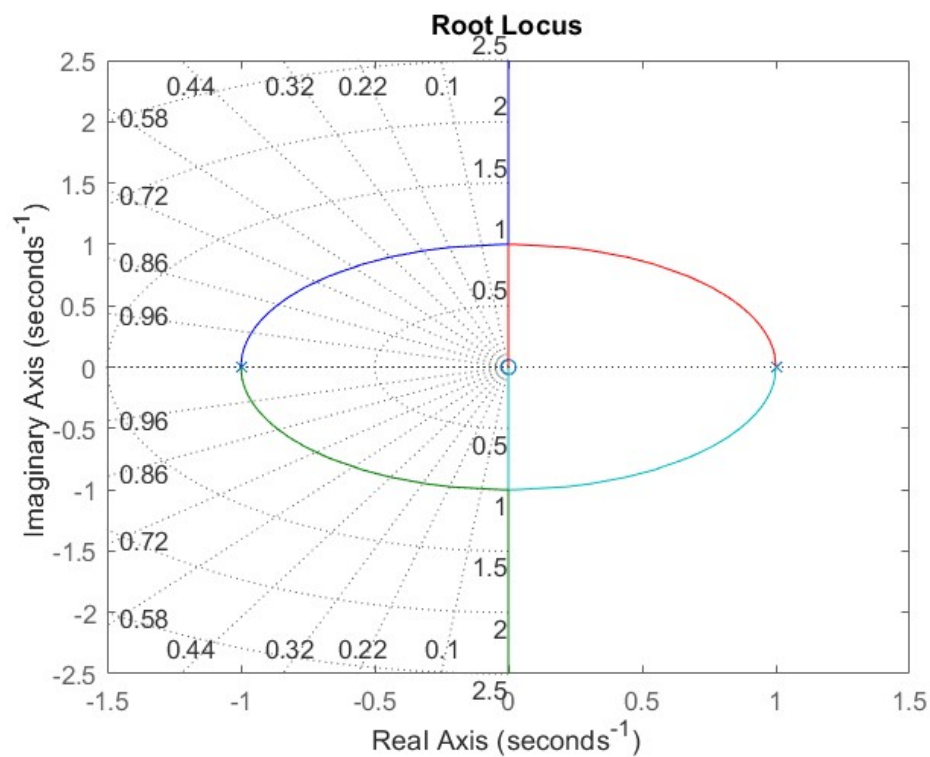


Figure 3: Root locus plot for $G_2(s)$.

```

HW4_question8_a.m
1  clear; clc; close all;
2
3  s = tf('s');
4
5  % Define G(s)
6
7  G = (s^2) / ((s^2 - 1)^2);
8
Command Window
0.0000 + 0.0000i
-1.0000 + 0.0000i
0.0000 + 1.0000i
0.0000 - 1.0000i
1.0000 + 0.0000i
fx >> |

```

Figure 4: Candidate breakaway/break-in points for $G_2(s)$.

2 c : Classification of Breakaway/Break-in Candidate Points

2.1 Problem Statement

Given a transfer function $G(s) = \frac{a(s)}{b(s)}$, manipulate the previously developed MATLAB function such that the candidate breakaway/break-in points are classified into the following categories:

1. Points that belong to the root locus for $K > 0$,
2. Points that belong to the complementary root locus for $K < 0$,
3. Candidate points that do not belong to either the root locus or its complementary locus.

The classification is to be verified using the root locus plot.

2.2 Methodology and Mathematical Background

For a unity-feedback system, the root locus is defined by the characteristic equation

$$1 + KG(s) = 0.$$

Solving for the gain yields

$$K(s) = -\frac{b(s)}{a(s)}.$$

Breakaway and break-in candidate points correspond to extrema of $K(s)$ on the real axis and therefore satisfy

$$\frac{dK}{ds} = 0.$$

Applying the quotient rule results in the condition

$$a(s)\frac{db(s)}{ds} - b(s)\frac{da(s)}{ds} = 0,$$

whose solutions define the set of candidate points.

To determine whether a real candidate point belongs to the root locus, the real-axis rule is applied. For $K > 0$, a real-axis point lies on the root locus if the total number of real poles and zeros at or to the right of that point is odd. For $K < 0$, the point belongs to the complementary root locus if this number is even. Any candidate point that is not real does not lie on either locus and is classified separately.

2.3 MATLAB implementation

The following MATLAB code computes the candidate points, applies the real-axis test, and classifies each point accordingly. The root locus is plotted for verification.

```
clear; clc; close all;

s = tf('s');

% define G(s)
G = 1 / ( s*(s+2)*(s+3) );

[RL_pts, CRL_pts, Other_pts, candidates] = breakaway_classify(G);

disp('All candidate points:'); disp(candidates)
disp('On Root Locus (K>0):'); disp(RL_pts)
disp('On Complementary RL (K<0):'); disp(CRL_pts)
disp('Not on RL or complementary:'); disp(Other_pts)

% root locus verification
figure; rlocus(G); grid on;

function [RL_pts, CRL_pts, Other_pts, candidates] = breakaway_classify(G)
    G = tf(G);
    [num, den] = tfdata(G, 'v');

    syms s
    a = poly2sym(num,s);
    b = poly2sym(den,s);

    eq = a*diff(b,s) - b*diff(a,s);
```

```

candidates = roots(sym2poly(eq));

tol = 1e-7;
z = zero(G); p = pole(G);
zR = real(z(abs(imag(z)) < tol));
pR = real(p(abs(imag(p)) < tol));

RL_pts = []; CRL_pts = []; Other_pts = [];

for i = 1:numel(candidates)
    c = candidates(i);

    if abs(imag(c)) > tol
        Other_pts(end+1,1) = c;
        continue
    end

    x = real(c);
    nRight = sum(zR >= x - tol) + sum(pR >= x - tol);

    if mod(nRight,2) == 1
        RL_pts(end+1,1) = x;
    else
        CRL_pts(end+1,1) = x;
    end
end

isBad = abs(imag(candidates)) > tol;
isBadReal = ~isBad & ~ismembertol(real(candidates), [RL_pts; CRL_pts], 1e-6);
Other_pts = [Other_pts; candidates(isBadReal)];
end

```

The function first determines all breakaway/break-in candidate points by solving the derivative condition derived from the gain expression. Each candidate is then examined to determine whether it is real. Non-real points are immediately excluded from the root locus analysis. For real points, the number of real poles and zeros at or to the right of the point is counted, and the parity of this count is used to classify the point as belonging to the root locus, the complementary root locus, or neither. The root locus plot serves as a visual confirmation of the classification.

2.4 Results

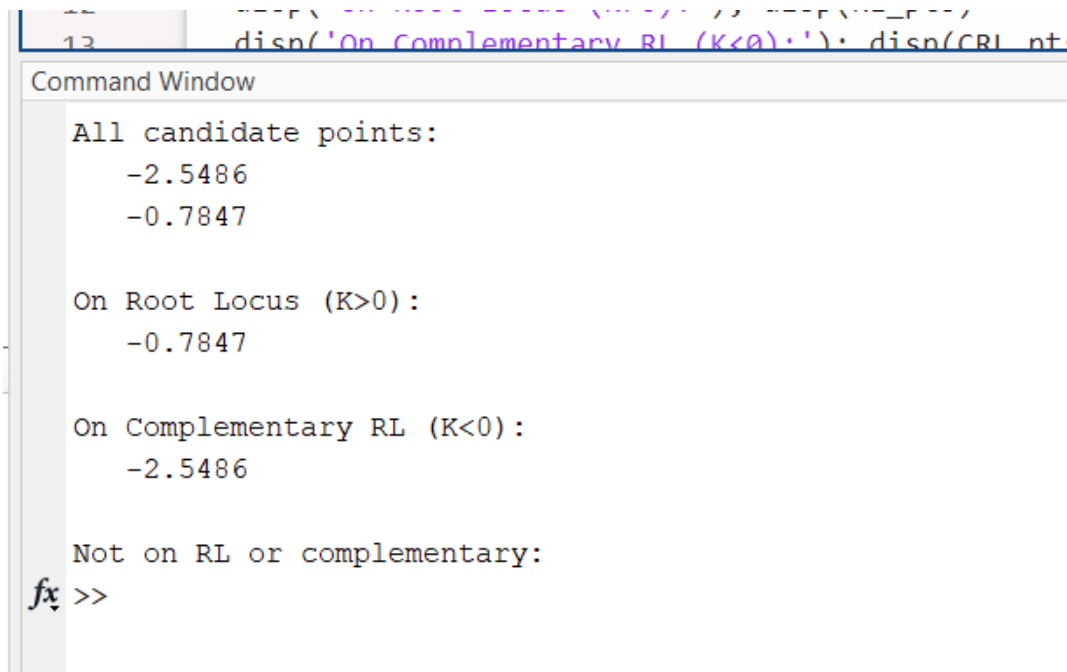


Figure 5: Root locus plot and classified candidate points .

2.5 (b) Plotting the Complementary Root Locus in MATLAB

Explain how the complementary root locus corresponding to negative gain values ($K < 0$) can be plotted in MATLAB using standard root locus functions, without relying on any built-in complementary root locus commands.

2.5.1 Theoretical Explanation

The root locus of a unity-feedback system is defined by the characteristic equation

$$1 + KG(s) = 0.$$

For the complementary root locus, the gain is negative. Letting $K = -K'$ with $K' > 0$, the characteristic equation becomes

$$1 - K'G(s) = 0.$$

This equation can be rewritten as

$$1 + K'(-G(s)) = 0.$$

Hence, the complementary root locus of $G(s)$ for $K < 0$ is equivalent to the standard root locus of the modified transfer function $-G(s)$ for positive gain values.

2.5.2 MATLAB implementation

Using the result above, the complementary root locus can be obtained by simply negating the original transfer function and applying MATLAB's standard `rlocus` command. The following code illustrates this procedure:

```
clear; clc; close all;

s = tf('s');

% define g
G = 1 / ( s*(s+2)*(s+3) );

% k > 0
figure;
rlocus(G);
grid on;
title('Root Locus (K > 0)');

% Complementary root locus for k < 0
figure;
rlocus(-G);
grid on;
title('Complementary Root Locus (K < 0)');
```

By negating the transfer function, the effect of negative gain values in the original system is transformed into positive gain values for the modified system. MATLAB's `rlocus` function then computes the complementary root locus using its standard algorithm. This approach preserves the mathematical definition of the complementary root locus while avoiding the use of any specialized or direct complementary root locus functions.

2.5.3 Result

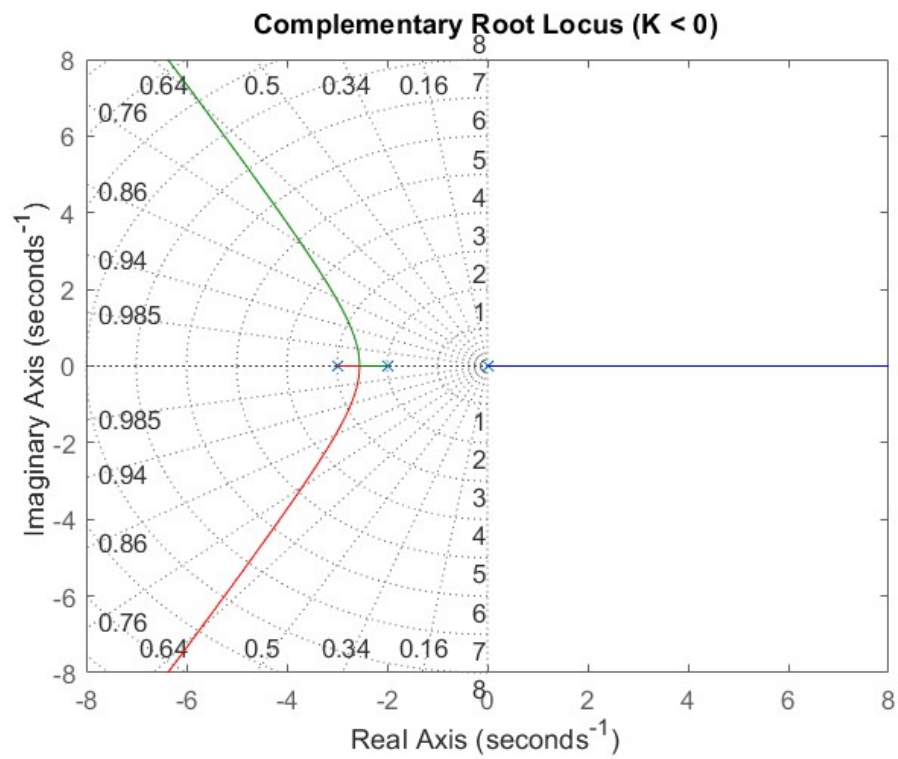


Figure 6: Complementary root locus obtained using `rlocus(-G)` .